

MBS-fast Manual

July 15, 2026

Contents

1	Build	2
1.1	Requirements	2
1.2	CPU build	2
1.3	GPU build	2
1.4	EPYC Zen5, AVX-512, and AVX2	3
1.5	Legacy root Makefile	3
2	Quick runs	3
3	Physical model	4
3.1	Pipeline	4
3.2	Ray Jones matrices	4
3.3	Dyadic rotation into the scattering basis	4
3.4	Kirchhoff diffraction integral	5
3.5	Coherent and incoherent Mueller accumulation	5
3.6	Orientation averaging	5
3.7	Scattering-angle weights	6
4	Particles and sizes	6
5	Orientation grids	6
6	Scattering grids	7
6.1	Unified convergence criterion	8
6.2	Adaptive configuration file	8
6.3	Why column symmetry does not divide Nphi	9
7	GPU and multi-GPU	9
7.1	CUDA backend	9
7.2	Why the GPU version is faster	9
7.3	Atomic and no-atomic GPU accumulation	10
7.4	Multi-size scans	11
8	All flags	12
8.1	Method, particle, and physical parameters	12
8.2	Orientation flags	12
8.3	Scattering grid and acceleration flags	12
8.4	Output, diagnostics, and legacy flags	13
9	Environment variables	14
10	Output	14

MBS-fast computes light scattering by non-spherical particles with geometrical ray tracing plus Physical Optics (PO) diffraction. The code is optimized for CPU OpenMP/MPI runs and CUDA GPU diffraction runs. This manual describes the current command line interface, build targets, grids, multi-GPU scans, and the main diffraction/Jones/Mueller formulas used by the implementation.

1 Build

1.1 Requirements

Component	Required for	Notes
GCC >= 9 or Clang >= 14	CPU and host CUDA code	GCC is the normal path on EPYC
OpenMP	CPU parallelism	GCC links with libgomp
MPI	cpu/ split build	mpicxx is used when available
CUDA toolkit	gpu/ split build	nvcc, libcudart, libcufft
NVIDIA driver	GPU runs	Must support the requested sm_XX binary

The split build is the recommended build path. CPU objects are kept under `cpu/build/`; CUDA objects are kept under `gpu/build/`. The shared physics code remains in `src/`.

1.2 CPU build

```
# MPI + OpenMP CPU binary
make -C cpu -j
```

```
# Run one MPI rank with 64 OpenMP threads
OMP_NUM_THREADS=64 cpu/bin/mbs_po_mpi --po -p 1 125.9 78.09 \
  --ri 1.3116 0 -w 0.532 -n 12 --oldauto 2 \
  --grid 0 180 600 180 --threads 64 --close -o out_cpu
```

```
# Run four MPI ranks, each with 16 OpenMP threads
mpirun -np 4 cpu/bin/mbs_po_mpi --po -p 1 125.9 78.09 \
  --ri 1.3116 0 -w 0.532 -n 12 --oldauto 2 \
  --grid 0 180 600 180 --threads 16 --close -o out_cpu_mpi
```

Debug help build:

```
make -C cpu debug
cpu/bin/mbs_po_mpi_debug --help-debug
```

1.3 GPU build

```
# Default GPU binary: float + fast math
PATH=/usr/local/cuda/bin:$PATH make -C gpu -j
```

```
# Explicit variants
make -C gpu float -j # gpu/bin/mbs_po_gpu_float
make -C gpu float_fast -j # gpu/bin/mbs_po_gpu_float_fast
make -C gpu double_fast -j # gpu/bin/mbs_po_gpu_double_fast
```

Target	Binary	CUDA precision	Fast math	Typical use
make -C gpu	gpu/bin/mbs_po_gpu_float_fast	FP32	yes	Fast production scans
make -C gpu float	gpu/bin/mbs_po_gpu_float	FP32	no	FP32 check without fast math
make -C gpu double_fast	gpu/bin/mbs_po_gpu_double_fast	FP64	yes	Reference GPU checks
make -C gpu double_debug	gpu/bin/mbs_po_gpu_double_debug	FP64	yes	Debug help and diagnostics

The GPU split build enables CUDA diffraction by default. `--gpu` is accepted but optional. Use `--cpu` only when you deliberately want the CPU diffraction backend from a GPU-capable binary.

The Makefile detects the GPU compute capability with `nvidia-smi`. Override when needed:

```
# Ampere, e.g. RTX 3080 Ti
make -C gpu double_fast -j GPU_ARCH=86
```

```
# Ada, e.g. RTX 4070
make -C gpu float_fast -j GPU_ARCH=89
```

1.4 EPYC Zen5, AVX-512, and AVX2

scripts/detect_arch_flags.sh is used by the Makefiles through ARCH_FLAGS. On a native machine, the safest high-performance option is usually:

```
make -C cpu clean
make -C cpu -j ARCH_FLAGS="-march=native -mtune=native"
```

```
make -C gpu clean
make -C gpu double_fast -j ARCH_FLAGS="-march=native -mtune=native"
```

For named AMD targets:

CPU target	GCC/Clang flags	Notes
EPYC Zen2	ARCH_FLAGS="-march=znver2 -mtune=znver2"	AVX2/FMA path
EPYC Zen3	ARCH_FLAGS="-march=znver3 -mtune=znver3"	AVX2/FMA path
EPYC Zen4	ARCH_FLAGS="-march=znver4 -mtune=znver4"	AVX-512 capable on GCC versions that support it
EPYC Zen5	ARCH_FLAGS="-march=znver5 -mtune=znver5"	Use with a compiler that knows znver5
Portable AVX2	ARCH_FLAGS="-O3 -mavx2 -mfma"	Runs on AVX2 machines
Explicit AVX-512	ARCH_FLAGS="-O3 -mavx512f -mavx512dq -mavx512cd -mavx512bw -mavx512vl"	Use only on machines that support these flags

There is no GCC/Clang option named AVX12. If the intended target is Zen5 with AVX-512, use -march=znver5 when the compiler supports it. If the compiler is older, either upgrade GCC/Clang or use the explicit AVX-512 flags above after checking `lscpu | grep avx512`.

1.5 Legacy root Makefile

The root Makefile still exists for compatibility:

```
make # bin/mbs_po, CPU
make gpu_double_fast # delegates to gpu/double_fast
make gpu_float_fast # delegates to gpu/float_fast
make USE_CUDA=1 # legacy single-tree CUDA build
```

Prefer the split cpu/ and gpu/ builds for normal work.

2 Quick runs

```
# GPU double, oldauto orientation grid, full 0..180 degree output
gpu/bin/mbs_po_gpu_double_fast --po -p 1 125.9 78.09 \
  --ri 1.3116 0 -w 0.532 -n 12 --oldauto 2 \
  --grid 0 180 600 180 --threads 64 --close -o hex_gpu_double
```

```
# Same run, force CPU backend from the GPU binary
gpu/bin/mbs_po_gpu_double_fast --po --cpu -p 1 125.9 78.09 \
  --ri 1.3116 0 -w 0.532 -n 12 --oldauto 2 \
  --grid 0 180 600 180 --threads 64 --close -o hex_cpu_from_gpu_bin
```

```
# File particle scaled by equivalent size parameter
gpu/bin/mbs_po_gpu_float_fast --po --pf particle.dat --k_eq 58.81 \
  --ri 1.6 0.002 -w 1.064 -n 14 --oldauto 2 --pole \
  --grid 0 180 600 180 --threads 16 --close -o particle_keq

# Two-point forward/backward check
gpu/bin/mbs_po_gpu_double_fast --po -p 1 125.9 78.09 \
  --ri 1.3116 0 -w 0.532 -n 1 --oldauto 2 --pole \
  --grid 0 180 600 1 --threads 64 --close -o check_0_180
```

3 Physical model

3.1 Pipeline

Stage	Main code path	Result
Particle setup	src/particle, src/main.cpp	Facets, normals, area, volume, symmetry
Geometrical tracing	src/scattering, src/tracer, src/Splitting.cpp	Output beams with direction, polygon, area, optical path, Jones matrix
PO diffraction	HandlerPO::ApplyDiffraction*, Handler::DiffractIncline*, CUDA equivalents	Far-field Jones amplitude per beam and direction
Coherent sum	HandlerPO::AddToMueller, GPU fused Mueller kernels	Sum Jones matrices over beams for one orientation
Mueller conversion	src/math/Mueller.cpp	4x4 Mueller matrix
Orientation average	HandlerPOTotal, TracerPOTotal	Final randomly oriented Mueller matrix

3.2 Ray Jones matrices

Each traced beam carries a 2x2 complex Jones matrix beam.J. Refraction and reflection multiply it by Fresnel coefficients in Splitting.cpp. The two polarization channels are the local vertical/horizontal basis of the incident and outgoing ray.

For a beam with local Jones matrix

$$J = \begin{bmatrix} J_{vv} & J_{vh} \\ J_{hv} & J_{hh} \end{bmatrix},$$

the code treats J as the coherent amplitude transform accumulated along the ray path. Cutoffs based on $|J|^2$, area, or $|J|^2 \cdot \text{area}$ can remove weak beams before expensive diffraction.

3.3 Dyadic rotation into the scattering basis

Before adding a beam to a detector direction, the beam-local Jones matrix must be expressed in the detector polarization basis. HandlerPO::RotateJones and RotateJonesFast build this 2x2 rotation/projection using dot products between:

Vector	Meaning
beam.Direction()	Beam propagation direction after tracing
direction	Requested far-field scattering direction
vf	Forward reference direction
info.polarization_vectors	Precomputed beam polarization basis

The effective far-field Jones contribution has the structure used in ApplyDiffractionFast:

$$J_{\text{beam}}(\theta, \phi) = F_{\text{edge}}(\theta, \phi) * R_{\text{out}}(\theta, \phi) * F_{\text{n}}(\theta, \phi)$$

where:

Factor	Code	Meaning
F_edge	DiffractionInclineFast, DiffractionIncline, DiffractionInclineAbs	Scalar Kirchhoff edge diffraction integral for the beam polygon
R_out	RotateJones or KarczewskiJones	Jones-basis rotation/projection from beam coordinates to scattering coordinates
F_n	ComputeFnJones	Fresnel/Jones correction for the beam normal and direction

The optional `--karczewski` flag replaces the default rotation with the Karczewski polarization matrix. The code comments note that M11 is unchanged because the Frobenius norm is preserved, while polarization-sensitive elements can change.

3.4 Kirchhoff diffraction integral

For each traced output beam the illuminated aperture is a polygon. The PO far field is evaluated by a boundary/edge form of the Kirchhoff integral. In code this lives in:

Function	Use
Handler::DiffractionIncline	CPU scalar non-absorbing edge integral
Handler::DiffractionInclineFast	CPU optimized edge integral with precomputed edge data
Handler::DiffractionInclineAbs	Absorbing variant
GpuDiffraction.cu kernels	CUDA implementation of the same aperture/edge calculation

Conceptually the scalar diffraction factor is the aperture phase integral

$$F(q) = \int_A \exp(i \mathbf{k} \cdot \mathbf{q} \cdot \mathbf{r}) dA,$$

where A is the projected beam polygon, $k = 2\pi/\lambda$, and q is the difference between the outgoing beam direction and the requested far-field direction. The implementation evaluates the polygon integral through its edges to avoid sampling the aperture area directly. For an edge from vertex a to b, the contribution is a stable complex exponential difference divided by the corresponding phase denominator; special small-denominator branches use sinc-like limits.

The GPU and CPU branches should use the same geometry, phase convention, and theta grid. Differences normally come from precision (float vs double), fast math, FFT interpolation, atomics/reduction order, and special pole or endpoint handling.

3.5 Coherent and incoherent Mueller accumulation

Default PO mode is coherent per orientation:

```
J_total(theta, phi) = sum_beams J_beam(theta, phi)
M(theta, phi)       = Mueller(J_total(theta, phi))
```

With `--incoh`, the conversion happens before beam summation:

```
M(theta, phi) = sum_beams Mueller(J_beam(theta, phi))
```

The Jones-to-Mueller conversion is implemented in `src/math/Mueller.cpp`. For Jones elements S1..S4, the code computes the standard 4x4 real Mueller matrix from bilinear products such as $|S1|^2$, $|S2|^2$, $\text{Re}(S1 \text{ conj}(S2))$, and $\text{Im}(S1 \text{ conj}(S2))$.

3.6 Orientation averaging

For randomly oriented particles, each orientation produces a Mueller matrix on the scattering grid. HandlerPOTotal and TracerPOTotal average orientations with the beta/gamma weights selected by the orientation mode. At beta poles, `--pole` uses one gamma value because all gamma rotations are equivalent at the exact pole. In current code, `--pole` keeps beta endpoints instead of midpoint beta sampling, so the beta pole is actually present in the grid.

3.7 Scattering-angle weights

Uniform theta grids output $N_{\text{th}} + 1$ rows. $2\pi \cdot d\cos$ is the solid-angle ring weight assigned to the theta row:

$$d\Omega(\theta_j) = 2\pi \cdot (\cos(\theta_{\text{left}}) - \cos(\theta_{\text{right}}))$$

For endpoint rows the interval is half-width and clipped to the requested range. For a full $0 \dots 180$ grid the sum of $2\pi \cdot d\cos$ over all rows is 4π .

4 Particles and sizes

Flag	Arguments	Description
-p	TYPE PARAMETERS...	Built-in particle. Exactly one of -p or --pf is required.
--pf	FILE	Load particle from file.
--rs	SIZE	Resize file particle to $D_{\text{max}} = \text{SIZE}$. Mutually exclusive with --k_eq.
--k_eq	X	Resize so $k_{\text{eq}} = 2\pi \cdot r_{\text{eq}} / \lambda$. Requires wavelength.
--ri	Re Im	Complex refractive index. Absorption is enabled automatically when $\text{Im} \neq 0$, or explicitly with --abs.
-w	LAMBDA	Wavelength in micrometers.
-n	N	Maximum internal reflection/refraction depth.

Built-in particle types:

Type	Shape	Parameters
1	Hexagonal column/plate	L D
2	Bullet	L D
3	Bullet rosette	L D [cap]
4	Droxtal	SCALE; legacy L D SCALE ignores L, D
10	Concave hexagonal	L D CAVITY_DEG; cavities must not intersect
12	Two-column hexagonal aggregate	L D 2
999	Fixed built-in aggregate	SCALE

Equivalent-size scaling:

$$r_{\text{eq}} = (3 V / (4\pi))^{(1/3)}$$

$$k_{\text{eq}} = 2\pi \cdot r_{\text{eq}} / \lambda$$

$$\text{scale} = (k_{\text{eq_target}} \cdot \lambda / (2\pi)) / r_{\text{eq_original}}$$

5 Orientation grids

Mode	Arguments	Best use	Notes
--oldauto	DIV	Production regular grid	Grid step follows diffraction-limited angular scale; common values are 2, 4, 8.
--random	Nb Ng	Manual beta/gamma regular grid	Uses symmetry-reduced beta/gamma domain unless overridden.
--fixed --orientfile	BETA GAMMA FILE	Single-orientation debugging Reproducible custom orientations	Angles are degrees. One beta/gamma pair per line.
--sobol	N	Quasi-random orientation average	Good convergence for broad scans.

Mode	Arguments	Best use	Notes
--sobol_seed	N S	Seeded Sobol/Owen sequence	Useful for repeatable convergence checks.
--sobol_ring	Nb Ng	Sobol beta with uniform gamma rings	Hybrid orientation grid.
--so3_quat	N	Full SO(3) Hammersley quaternions	Does not rely on beta/gamma symmetry domain.
--hammersley	N	Low-discrepancy orientations	Debug/experimental.
--lattice	N	Rank-1 lattice orientations	Debug/experimental.
--lattice_z	N Z	Rank-1 lattice with explicit generator	Debug/experimental.
--euler_quad	Nb Ng	Gauss in cos(beta), periodic gamma	High-order quadrature.
--euler_adapt	Nb NgMax	Adaptive gamma count per beta ring	Reduces work near beta poles.
--adaptive_euler	EPS	Converged regular Euler grid	Refines N_β and N_γ separately, then together.
--adaptive	EPS	Adaptive Sobol count only	Doubles N ; it does not silently alter theta, phi, or reflection depth.
--auto	EPS	Auto theta, phi, orientation count	Keeps the requested reflection depth fixed.
--autofull	EPS	Auto n, theta, phi, orientations	Slower, more complete search.
--oldautofull	EPS	Autofull with converged Euler grid	Independently converges N_β and N_γ .
--adaptive-config	FILE	Reproducible adaptive settings	Per-parameter minimum, maximum, and tolerance.

Orientation modifiers:

Flag	Arguments	Description
--ring_points	N	Points per diffraction ring for oldauto estimates; default 3.
--mirror_gamma	none	Halve the gamma symmetry range and mirror the Mueller result.
--sym	Sb Sg	Override symmetry domain: beta range is pi/Sb, gamma range is 2*pi/Sg.
--b	B1 B2	Manual beta range in degrees for --random.
--g	G1 G2	Manual gamma range in degrees for --random.
--maxorient	N	Maximum orientations for adaptive modes.
--max_beta_points	N	Hard adaptive N_β limit.
--max_gamma_points	N	Hard adaptive N_γ limit.
--chunk	N	Orientation/gamma chunk size.
--coh_orient	none	Legacy coherent-across-orientations mode.
--pole	none	Use one gamma value at beta poles. Best with endpoint grids such as --oldauto.
--owen_avg	K	Average K nested Owen final seeds in --autofull; default can be controlled by environment.
--owen_seeds	S...	Explicit Owen seeds for --autofull final averaging.

6 Scattering grids

Flag	Arguments	Description
--grid	T1 T2 Nphi Nth	Uniform theta grid from T1 to T2 degrees. Output has Nth + 1 theta rows.
--grid	R Nphi Nth	Backscatter cone grid of radius R degrees.
--tgrid	FILE	Non-uniform theta grid in degrees, one row per theta value.

Flag	Arguments	Description
--auto_tgrid	EPS	Adaptive theta grid by bisection of measured full-Mueller interpolation error; no fixed halo angles are used.
--auto_phi	none	Compatibility form of measured phi convergence with tolerance 0.02.
--adaptive_phi	EPS	Converge N_ϕ only.
--	EPS	Converge internal reflection depth n only.
adaptive_reflections		
--nphi	N	Override phi count. Highest priority for phi.
--max_theta_points	N	Hard theta-point limit; default 4097.
--max_phi_points	N	Hard N_ϕ limit; default 2400.
--convergence_passes	N	Consecutive passing refinements required; default 2.
--filter	DEG	Restrict output to a backscattering cone. Legacy/debug use.
--point	none	Backscatter point mode. Legacy; avoid for optimized regular-grid production.

Selection rules:

Quantity	Rule
Theta grid	Exactly one of --tgrid, --grid, or --auto_tgrid; unified auto modes treat an explicit grid as fixed.
Phi grid	Standalone adaptive phi selection and explicit --nphi are mutually exclusive. Unified auto modes treat explicit --nphi as fixed and tune the remaining dimensions.

6.1 Unified convergence criterion

For two approximations A and B , phi, reflection, and Euler candidate comparisons use the row error

$$e_{\text{row}} = \max \left(\frac{|M_{11}^A - M_{11}^B|}{\max(|M_{11}^A|, |M_{11}^B|, \epsilon_0)}, \max_{(i,j) \neq (1,1)} \frac{|R_{ij}^A - R_{ij}^B|}{\max(1, |R_{ij}^A|, |R_{ij}^B|)} \right), \quad R_{ij} = \frac{M_{ij}}{M_{11}}.$$

Here M_{ij} is a Mueller element, R_{ij} is its normalized value, and ϵ_0 protects a vanishing M_{11} . The maximum over theta rows is combined with the relative change of $\int M_{11} d\Omega$. Reflection-depth tests also include outgoing energy. Theta convergence is different: it evaluates the actual Mueller row at one-third and two-thirds of every interval against interpolation and checks the same integrated M_{11} . Sobol orientation convergence checks incoming energy and all 16 Mueller elements on control-theta rows selected from the actual scattering grid.

The parameters are not strictly independent. Reflection depth changes the beam tree; phi changes the azimuth average used to expose theta structure; and orientation sampling changes every averaged row. Therefore the joint order is

$$n \longrightarrow N_\phi \longrightarrow \{\theta_j\} \longrightarrow N_{\text{orient}}.$$

The --auto modes repeat the complete sequence on a common larger pilot set until the pilot implied by the selected orientation count and all selected values form a coupled fixed point. Regular Euler convergence alternates N_β and N_γ , then doubles both together. Reaching any hard limit in phi, reflection, theta, Euler, or a unified mode is reported as an error rather than accepted as convergence. Standalone ---adaptive-orientations is exploratory: at its orientation cap it warns and writes the best result; unified modes use strict orientation convergence. The configured passing streak applies to phi, reflection, Euler, and orientation searches, while theta uses interval refinement directly. Every trial is written to <output>/<name>_convergence.tsv.

6.2 Adaptive configuration file

Use ---adaptive-config FILE to keep production convergence settings outside a long command line. The documented template is configs/adaptive.example.conf. It uses strict key = value assignments and accepts inline comments after # or ;.

The file defines separate minimum, maximum, and relative tolerance values for reflection depth, laboratory alpha (internally N_ϕ), theta points, Sobol orientations, beta points, and gamma points. It also defines the number of consecutive passing refinements and the pilot and sweep limits of the coupled controller. A tolerance of 0.01 means one percent.

```
mbs_po --autofull 0.02 --adaptive-config configs/adaptive.example.conf ...
```

Per-parameter tolerances in the file replace the mode-wide EPS; a missing tolerance inherits EPS. Explicit command-line `---max-*`, `---max-reflections`, and `---convergence-passes` values override the file. Unknown keys, duplicates, invalid ranges, non-power-of-two Sobol limits, and malformed lines are rejected before the calculation, with the file line and a corrective hint.

6.3 Why column symmetry does not divide Nphi

The code uses the Euler convention

$$\mathbf{R} = R_z(\alpha)R_y(\beta)R_z(\gamma).$$

The scattering-azimuth loop is the rotationally covariant implementation of the laboratory-angle α average, so $N_\phi = N_\alpha$ in this context. A hexagonal column has the body-axis symmetry $R_z(2\pi/6)$; it reduces the γ domain to $0 \leq \gamma < \pi/3$. For $\beta \neq 0, \pi$, it does not make the tilted particle invariant under a laboratory $R_z(2\pi/6)$ rotation, hence $0 \leq \alpha < 2\pi$ is still required. An automatic division of N_ϕ by six would therefore be incorrect. The aliases `--alpha-points` and `--adaptive-alpha` only clarify the meaning of the existing phi controls; they do not enable an approximation.

For full angular integrals use `--grid 0 180 Nphi Nth`. Endpoint rows are physical half-cells; they must not have zero weight.

7 GPU and multi-GPU

7.1 CUDA backend

Flag	Description
<code>--gpu</code>	Use CUDA diffraction backend. Optional in <code>gpu/</code> binaries because they default to GPU.
<code>--cpu</code>	Force CPU backend from a GPU-capable binary.
<code>--fft</code>	Use cuFFT angular phi interpolation backend. Requires GPU.
<code>--threads N</code>	OpenMP host worker threads. Also controls tracing-side parallelism before GPU diffraction.

GPU runtime errors are fatal in the split GPU build. For debugging only:

```
MBS_GPU_ALLOW_FALLBACK=1 gpu/bin/mbs_po_gpu_float_fast ...
```

7.2 Why the GPU version is faster

The expensive PO part is the repeated evaluation of the same beam aperture integral for many detector directions and many orientations. The CPU traces rays and prepares compact beam records; the GPU then evaluates diffraction and, in the fast paths, Mueller accumulation on a large regular grid.

Work item	CPU path	GPU path	Parallel granularity
Ray tracing through facets	OpenMP over orientations/gamma blocks	Mostly still CPU-side; <code>--gpu_trace</code> is only a candidate prefilter	Orientation/chunk level
Beam packing	CPU host code	CPU prepares <code>GpuBeam/packed</code> beam buffers and copies to device	Beam records
Edge diffraction	Scalar/vector CPU loops	CUDA kernels in <code>GpuDiffraction.cu</code>	Beam x theta x phi x orientation

Work item	CPU path	GPU path	Parallel granularity
Jones coherent sum	CPU arrays of complex 2x2 matrices	Atomic or no-atomic device kernels	Grid cell and orientation
Jones to Mueller	CPU after Jones sum	Separate <code>mueller_batch_kernel</code> or fused in diffraction kernel	Grid cell
Multi-k_eq scans	Repeat diffraction per size	Optional fused multi-k_eq CUDA pass	Size x beam x grid
FFT phi interpolation	CPU direct phi grid unless disabled	cuFFT low-phi pass plus zero-padding reconstruction	Theta/orientation batches

The main speedup comes from exposing a very large product of independent operations:

$N_{\text{work}} \sim N_{\text{orient}} * N_{\text{beams_per_orientation}} * N_{\text{theta}} * N_{\text{phi}}$

Each beam/direction contribution is mostly independent until the coherent Jones sum. That makes the diffraction stage a good CUDA workload: many threads do the same edge-integral arithmetic over different (orientation, beam, theta, phi) combinations.

7.3 Atomic and no-atomic GPU accumulation

There are two families of CUDA accumulation paths in `src/cuda/GpuDiffraction.cu`.

Mode	Main kernels	How accumulation works	When useful
Atomic Jones accumulation	<code>diffraction_kernel</code>	Each thread computes one beam/grid contribution and uses <code>atomicAdd</code> into the complex Jones buffer. Afterwards <code>mueller_batch_kernel</code> converts the summed Jones matrix to Mueller.	General fallback, supports more combinations such as no-shadow output.
No-atomic grid kernels	<code>diffraction_grid_kernel</code>	Work is reorganized so a thread owns a grid/output slot and loops over beams, avoiding multiple writers to the same Jones cell.	Full-only output when memory/layout allows it.
Fused Mueller kernels	<code>diffraction_grid_mueller_kernel</code> , <code>diffraction_grid_mueller_full_kernel</code> , <code>*_full8_kernel</code> , <code>*_mixed8_kernel</code>	The kernel computes coherent Jones locally for an output slot and immediately converts/adds Mueller, avoiding large intermediate Jones buffers.	Fast production path for full-only GPU runs.
Staged orientation Mueller	<code>diffraction_grid_mueller_orient_kernel</code> + <code>reduce_mueller_orient_kernel</code>	First writes per-orientation Mueller, then reduces orientations.	Reduces contention and gives a deterministic reduction shape for large orientation batches.
Multi-k_eq fused	<code>diffraction_grid_mueller_multik_kernel</code>	Computes several nearby equivalent sizes from one packed beam batch.	Shared-batch --multikeq scans.

In the atomic path the atomics are on the real and imaginary components of the 2x2 Jones matrix:

`J00.re, J00.im, J01.re, J01.im,`
`J10.re, J10.im, J11.re, J11.im`

For `_noshadow` output the same set of atomics is also applied to the no-shadow Jones buffer. This is correct for coherent PO because beams must be summed as Jones amplitudes before conversion to Mueller. The cost is contention when many beams hit the same detector cell.

The no-atomic/fused paths avoid that contention by changing ownership: one CUDA thread or thread group owns an output grid element and accumulates the beams for that element in registers/local variables. This is often faster for full Mueller output because it reduces global-memory atomics and may skip the large intermediate Jones buffer. The tradeoff is that these paths are more specialized and may be disabled for diagnostic modes that require extra outputs.

Useful controls:

Environment variable	Effect
MBS_GPU_NO_ATOMICS=1	Force no-atomic path when supported.
MBS_GPU_NO_ATOMICS=0	Force atomic path for comparison/debugging.
MBS_GPU_FUSED_MUELLER=1	Prefer fused diffraction-to-Mueller kernels when no-atomic mode is active.
MBS_GPU_STAGE_MUELLER=1	Use staged per-orientation Mueller plus reduction where supported.
MBS_GPU_NO_VERTEX_CACHE=1	Disable cached/packed vertex path for debugging.
MBS_GPU_TIMING=1	Print GPU timing breakdown for count/pack/copy/kernels/d2h/add.
MBS_GPU_BLOCK=N	Override CUDA block size for kernel tuning.

Do not confuse this with full GPU ray tracing. The production GPU backend is primarily a diffraction and Mueller-accumulation accelerator. `--gpu_trace` only prefilters nonconvex tracing candidates; exact intersections remain CPU-side.

7.4 Multi-size scans

Flag	Arguments	Description
<code>--multigrid</code>	Dmin Dmax N	Log-spaced scan in absolute maximum particle dimension; each value uses $x = \pi D_{\max}/\lambda$, independent of the source size.
<code>--multikeq</code>	Kmin Kmax N	Log-spaced scan in equivalent size parameter.
<code>--multikeq_list</code>	FILE	Exact list of k_eq values, one per line.
<code>--multigrid_parallel</code>	N	Run scan points as child processes; 0 means auto.
<code>--multigrid_threads</code>	N	OpenMP threads per child process.
<code>--gpu_devices</code>	LIST	Comma-separated CUDA device list, for example 0,1,2,3,4.
<code>--multikeq_shared_batches</code>	none	Batch nearby k_eq values per GPU child and reuse tracing from the largest value in the batch.
<code>--multikeq_batch_ratio</code>	R	Maximum kmax/kmin inside one shared batch; default 1.05.

Exact multi-GPU scan, one process per active GPU slot:

```
gpu/bin/mbs_po_gpu_float_fast --po --pf particle.dat \
  --multikeq_list keq.txt --ri 1.6 0.002 -w 1.064 -n 12 \
  --oldauto 2 --grid 0 180 600 180 --fft --close \
  --multigrid_parallel 0 --multigrid_threads 16 --gpu_devices 0,1,2,3,4 \
  -o scan_exact
```

Shared-batch scan:

```
gpu/bin/mbs_po_gpu_float_fast --po --pf particle.dat \
  --multikeq_list keq.txt --ri 1.6 0.002 -w 1.064 -n 12 \
  --oldauto 2 --grid 0 180 600 180 --fft --close \
  --multigrid_parallel 0 --multigrid_threads 16 --gpu_devices 0,1,2,3,4 \
  --multikeq_shared_batches --multikeq_batch_ratio 1.05 \
  -o scan_shared
```

Experimental fused multi-k_eq GPU diffraction:

```
MBS_GPU_MULTI_K_FULL=1 gpu/bin/mbs_po_gpu_float_fast ...
```

Use tighter `--multikeq_batch_ratio` for accuracy; use exact mode when each size must have its own oldauto reference grid.

8 All flags

8.1 Method, particle, and physical parameters

Flag	Arguments	Description
--po	none	Physical Optics mode.
--go	none	Geometrical Optics mode.
-p	TYPE L D [extra]	Built-in particle.
--pf	FILE	Particle from file.
--rs	SIZE	Resize file particle by Dmax.
--k_eq	X	Resize by equivalent size parameter.
--ri	Re Im	Refractive index.
-w	LAMBDA	Wavelength in micrometers.
-n	N	Maximum internal reflections/refractions.
--abs	none	Enable absorption accounting. Also enabled automatically for nonzero Im(ri).
--abs_points	N or all	Absorption sampling: center point or all polygon vertices.

8.2 Orientation flags

Flag	Arguments	Description
--fixed	BETA GAMMA	Single orientation in degrees.
--random	Nb Ng	Regular beta/gamma grid.
--sobol	N	Sobol orientations.
--so3_quat	N	Full SO(3) quaternion orientations.
--sobol_seed	N S	Sobol with explicit Owen seed.
--sobol_ring	Nb Ng	Sobol beta with gamma rings.
--hammersley	N	Hammersley orientation set.
--lattice	N	Rank-1 lattice orientation set.
--lattice_z	N Z	Rank-1 lattice with explicit generator.
--euler_quad	Nb Ng	Gauss beta x periodic gamma quadrature.
--euler_adapt	Nb NgMax	Gauss beta with adaptive gamma count.
--adaptive_euler	EPS	Independently converge N_β and N_γ , then verify them jointly.
--montecarlo	N	Pseudo-random Monte Carlo orientations.
--adaptive	EPS	Adaptive Sobol orientation count only.
--auto	EPS	Auto theta, phi, and orientation count.
--autofull	EPS	Auto n, theta, phi, and orientations.
--oldautofull	EPS	Autofull search with a converged regular Euler final grid.
--adaptive-config	FILE	Load per-parameter adaptive ranges and tolerances.
--owen_avg	K	Average final Owen seeds.
--owen_seeds	S...	Explicit final Owen seeds.
--oldauto	DIV	Physics-based regular grid.
--ring_points	N	Points per diffraction ring estimate.
--mirror_gamma	none	Use mirrored half gamma domain.
--orientfile	FILE	Load beta/gamma orientations from file.
--b	B1 B2	Beta range for --random.
--g	G1 G2	Gamma range for --random.
--maxorient	N	Maximum adaptive orientation count.
--max_beta_points	N	Maximum adaptive N_β .
--max_gamma_points	N	Maximum adaptive N_γ .
--chunk	N	Chunk size for orientation/gamma processing.
--coh_orient	none	Legacy coherent orientation mode.
--pole	none	Exact beta-pole gamma shortcut.
--sym	Sb Sg	Override particle symmetry.

8.3 Scattering grid and acceleration flags

Flag	Arguments	Description
--grid	T1 T2 Nphi Nth	Uniform theta range.
--grid	R Nphi Nth	Backscatter cone.
--tgrid	FILE	Non-uniform theta grid.
--auto_tgrid	EPS	Adaptive theta grid.
--auto_phi	none	Measured phi convergence at tolerance 0.02.
--adaptive_phi	EPS	Converge N_ϕ only.
--	EPS	Converge n only.
adaptive_reflections		
--nphi	N	Override phi count.
--max_theta_points	N	Adaptive theta limit.
--max_phi_points	N	Adaptive phi limit.
--convergence_passes	N	Required passing streak.
--threads	N	OpenMP worker threads.
--gpu	none	CUDA backend.
--cpu	none	Force CPU backend in GPU build.
--fft	none	cuFFT phi interpolation.
--beam_cutoff	EPS	Set relative beam J and area cutoffs.
--beam_cutoff_j	EPS	Skip beams by relative squared Jones magnitude.
--beam_cutoff_area	EPS	Skip beams by relative beam area.
--beam_cutoff_importance	EPS	Skip beams by relative Jones intensity times area.
--trace_cutoff	EPS	Set trace pruning J and area cutoffs.
--trace_cutoff_j	EPS	Prune trace tree by relative squared Jones magnitude.
--trace_cutoff_area	EPS	Prune trace tree by relative area.
--trace_cutoff_importance	EPS	Prune trace tree by relative Jones intensity times area.
--trace_max_beams	N	Abort one orientation after N beam nodes; 0 disables.
--gpu_trace	none	Experimental CUDA prefilter for nonconvex tracing candidates.
--trace_prefilter	none	Enable CPU projected-AABB prefilter.
--no_trace_prefilter	none	Disable CPU projected-AABB prefilter.
--trace_prefilter_margin	M	AABB prefilter margin.
--	none	Print prefilter counters.
trace_prefilter_stats		
-r	RATIO	Beam area restriction ratio; default 100.

8.4 Output, diagnostics, and legacy flags

Flag	Arguments	Description
-o	NAME	Output path/name.
--close	none	Exit after computation.
--log	SEC	Progress logging interval.
--checkpoint	none	Save/resume long --orientfile, --oldauto, and --random runs.
--save_betas	none	Save per-beta Mueller files.
--full_only	none	Write only full Mueller output. This is the default.
--noshadow_output	none	Also write _noshadow Mueller output.
--shadow	none	Legacy flag; currently no effect.
--shadow_off	none	Disable shadow beam. Diagnostic use.
--incoh	none	Incoherent per-beam Mueller sum.
--jones	none	Write Jones matrices where supported.
--karczewski	none	Use Karczewski polarization matrix.
--legacy_sign	none	Use old Fresnel sign convention.
--ot_phase_avg	none	Average optical-theorem extinction over one far-reference phase period.
--ot_phase_shift	F	Diagnostic optical-theorem phase shift in wavelengths.
--ot_ping	D	Legacy far-screen optical-theorem phase rotation.
--filter	DEG	Restrict output to backscatter cone.
--point	none	Legacy backscatter point mode.
--tr	FILE	Load trajectory file.

Flag	Arguments	Description
--all	none	Calculate all loaded trajectories.
--gr	none	Output trajectory groups.
--forced_convex	none	Force convex processing.
--forced_nonconvex	none	Force nonconvex processing.
--help, -h	none	Print help.
--help-debug	none	Print full debug/experimental help.

9 Environment variables

Production-use variables:

Variable	Meaning
CUDA_VISIBLE_DEVICES	Standard CUDA device visibility control.
OMP_NUM_THREADS	OpenMP default thread count when --threads is not used.
MBS_GPU_ALLOW_FALLBACK=1	Allow old GPU-to-CPU fallback after CUDA failure. Debug only.
MBS_GPU_MEM_FRACTION	Fraction of free GPU memory allowed for internal buffers.
MBS_GPU_NO_ATOMICS	Select no-atomic (1) or atomic (0) GPU accumulation path when possible.
MBS_GPU_FUSED_MUELLER	Select fused diffraction-to-Mueller kernels when no-atomic mode is active.
MBS_GPU_STAGE_MUELLER	Enable staged per-orientation Mueller reduction when supported.
MBS_GPU_NO_VERTEX_CACHE	Disable cached/packed vertex path for debugging.
MBS_GPU_TIMING	Print CUDA timing breakdowns.
MBS_GPU_BLOCK	Override CUDA kernel block size.
MBS_HOST_MEM_FRACTION	Host-memory fraction for oldauto/random chunking.
MBS_HOST_MEM_RESERVE_MB	Host memory reserve in MB.
MBS_HOST_MEM_BUDGET_MB	Hard host memory budget, also set for parallel children.
MBS_FFT_PHI_FACTOR	Override FFT reduced phi factor.
MBS_GPU_MULTI=0	Disable automatic multi-orientation GPU batching.
MBS_GPU_MULTI_MAX=N	Cap automatic GPU multi batching.
MBS_GPU_MULTI_K_FULL=1	Experimental fused multi-k_eq diffraction.
MBS_BEAM_LOG=PATH	Write the first handled PO beam set to the explicit diagnostic path. No beam log is written by default.
MBS_SHARED_BETA_GROUP=N	Override shared-batch beta grouping.
MBS_SHARED_ORIENT_CHUNK=N	Override shared-batch orientation chunk size.
MBS_PARALLEL_MEM_FRACTION	Memory fraction used by parallel multi-size scheduler.
MBS_PARALLEL_SHARED_KEQ=1	Environment equivalent of shared k_eq batching.
MBS_PARALLEL_KEQ_BATCH_RATIO=R	Environment default for shared batch ratio.

Diagnostic variables exist for FFT refinement, autofull heuristics, trace prefilters, and internal debug ranges. They are intentionally not recommended for normal production runs; inspect `grep -R "getenv(\"MBS_\" src` when debugging a specific subsystem.

10 Output

Main output is a whitespace-separated .dat file:

```
ScAngle 2pi*dcos M11 M12 M13 M14 M21 M22 M23 M24 M31 M32 M33 M34 M41 M42 M43 M44
```

Column	Meaning
ScAngle	Scattering angle theta in degrees.
2pi*dcos	Solid-angle ring weight for this theta row.
Mij	Mueller matrix elements after beam and orientation averaging.

Common companion outputs:

Output	Created by	Meaning
_noshadow file	- -noshadow_output	Mueller matrix without shadow/external beam contribution.
_betas/ directory	- -save_betas	Per-beta Mueller diagnostics.
checkpoint files	- -checkpoint	Resume data for long orientation-grid runs.
Jones output	- -jones	Raw Jones matrices in supported PO modes.
particle_for_check.dat	every calculation	Effective resized particle, stored inside the result directory.

Normal runs do not create diagnostic files in the current working directory. Use MBS_BEAM_LOG=PATH only when an explicit first-beam dump is needed.

Warnings about optical-theorem/integral mismatch are diagnostics. For non-absorbing runs the physical absorption is fixed to zero; the printed integral characteristic is a consistency check, not a replacement for the output Mueller matrix.